Question 1,2 and 3

```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules


df = pd.read_csv("Dataset_Day15.csv", header=None)
transactions = df.apply(lambda row: [item for item in row if pd.notnull(item) and item != ''], axis=1)

print(f" Original dataset shape: {df.shape}")
print(f" Sample cleaned transactions:\n{transactions.head().tolist()}")

#  Encode transactions
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)
print("\n Transactions encoded for market basket analysis.")

#  Find frequent itemsets (min_support=0.02)
frequent_itemsets = apriori(df_encoded, min_support=0.02, use_colnames=True)
```

```
 Original dataset shape: (7501, 20)
 Sample cleaned transactions:
[['shrimp', 'almonds', 'avocado', 'vegetables mix', 'green grapes', 'whole weat flour', 'yams', 'cottage cheese', 'energy drink', 'tomato juice', 'low fat yogurt', 'green tea', 'honey', 'salad', 'mineral water', 'salmon', 'antiox
ydant juice', 'frozen smoothie', 'spinach', 'olive oil'], ['burgers', 'meatballs', 'eggs'], ['chutney'], ['turkey', 'avocado'], ['mineral water', 'milk', 'energy bar', 'whole wheat rice', 'green tea']]

 Transactions encoded for market basket analysis.
   asparagus  almonds  antioxydant juice  asparagus  ...  whole wheat rice   yams  yogurt cake  zucchini
0      False     True               True      False  ...             False   True        False     False
1      False    False              False      False  ...             False  False        False     False
2      False    False              False      False  ...             False  False        False     False
3      False    False              False      False  ...             False  False        False     False
4      False    False              False      False  ...              True  False        False     False

[5 rows x 120 columns]
[' asparagus', 'almonds', 'antioxydant juice', 'asparagus', 'avocado', 'babies food', 'bacon', 'barbecue sauce', 'black tea', 'blueberries', 'body spray', 'bramble', 'brownies', 'bug spray', 'burger sauce', 'burgers', 'butter', '
cake', 'candy bars', 'carrots', 'cauliflower', 'cereals', 'champagne', 'chicken', 'chili', 'chocolate', 'chocolate bread', 'chutney', 'cider', 'clothes accessories', 'cookies', 'cooking oil', 'corn', 'cottage cheese', 'cream', 'd
essert wine', 'eggplant', 'eggs', 'energy bar', 'energy drink', 'escalope', 'extra dark chocolate', 'flax seed', 'french fries', 'french wine', 'fresh bread', 'fresh tuna', 'fromage blanc', 'frozen smoothie', 'frozen vegetables',
ate', 'chocolate bread', 'chutney', 'cider', 'clothes accessories', 'cookies', 'cooking oil', 'corn', 'cottage cheese', 'cream', 'dessert wine', 'eggplant', 'eggs', 'energy bar', 'energy drink', 'escalope', 'extra dark chocolate'
, 'flax seed', 'french fries', 'french wine', 'fresh bread', 'fresh tuna', 'fromage blanc', 'frozen smoothie', 'frozen vegetables', 'gluten free bar', 'grated cheese', 'green beans', 'green grapes', 'green tea', 'ground meat', 'g
ums', 'ham', 'hand protein bar', 'herb & pepper', 'honey', 'hot dogs', 'ketchup', 'light cream', 'light mayo', 'low fat yogurt', 'magazines', 'mashed potato', 'mayonnaise', 'meatballs', 'melons', 'milk', 'mineral water', 'mint',
'mint green tea', 'muffins', 'mushroom cream sauce', 'napkins', 'nonfat milk', 'oatmeal', 'oil', 'olive oil', 'pancakes', 'parmesan cheese', 'pasta', 'pepper', 'pet food', 'pickles', 'protein bar', 'red wine', 'rice', 'salad', 's
almon', 'salt', 'sandwich', 'shallot', 'shampoo', 'shrimp', 'soda', 'soup', 'spaghetti', 'sparkling water', 'spinach', 'strawberries', 'strong cheese', 'tea', 'tomato juice', 'tomato sauce', 'tomatoes', 'toothpaste', 'turkey', 'v
egetables mix', 'water spray', 'white wine', 'whole weat flour', 'whole wheat pasta', 'whole wheat rice', 'yams', 'yogurt cake', 'zucchini']

 Total frequent itemsets found: 103
```

Question 4

```python
# Step 4: Generate association rules (min confidence = 15%)
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.15)
print(f" Total association rules generated: {len(rules)}")

# 4a: Top 5 rules by lift
top_lift = rules.sort_values(by='lift', ascending=False).head(5)
print("\n Top 5 rules by LIFT:")
print(top_lift[['antecedents', 'consequents', 'lift']])

#  4b: Top 5 rules by leverage
top_leverage = rules.sort_values(by='leverage', ascending=False).head(5)
print("\n Top 5 rules by LEVERAGE:")
print(top_leverage[['antecedents', 'consequents', 'leverage']])
```

```
 Total association rules generated: 71

 Top 5 rules by LIFT:
     antecedents          consequents      lift
52    (spaghetti)        (ground meat)  2.291162
53  (ground meat)          (spaghetti)  2.291162
67    (olive oil)          (spaghetti)  1.999758
62         (soup)      (mineral water)  1.914955
41         (milk)  (frozen vegetables)  1.910382

 Top 5 rules by LEVERAGE:
       antecedents      consequents  leverage
53    (ground meat)      (spaghetti)  0.022088
52      (spaghetti)    (ground meat)  0.022088
64      (spaghetti)  (mineral water)  0.018223
63  (mineral water)      (spaghetti)  0.018223
50  (mineral water)    (ground meat)  0.017507
```

```python
#  Zhang's metric
rules['zhang'] = rules.apply(
    lambda x: ((x['support'] * (1 - x['lift'])) /
    ((x['support'] - x['confidence'] * x['consequent support']) + 1e-10))
    if x['lift'] != 1 else 0,
    axis=1
)

top_zhang = rules.sort_values(by='zhang', ascending=False).head(2)
bottom_zhang = rules.sort_values(by='zhang', ascending=True).head(2)

print("\nTop 2 rules by Zhang's metric:")
print(top_zhang[['antecedents', 'consequents', 'zhang']])

print("\n Bottom 2 rules by Zhang's metric:")
print(bottom_zhang[['antecedents', 'consequents', 'zhang']])
```

```
Top 2 rules by Zhang's metric:
    antecedents     consequents     zhang
19  (chocolate)     (spaghetti)  5.968656
10  (chocolate)  (french fries)  5.293616

 Bottom 2 rules by Zhang's metric:
       antecedents  consequents     zhang
18     (spaghetti)  (chocolate) -6.342607
9   (french fries)  (chocolate) -5.521900
```